

CHAPITRE VIII

FONCTIONS AVANCEES

I. INTRODUCTION

Cette unité est une extension de l'unité précédente, où différents opérateurs et fonctions ont été discutés en détail. Cet appareil traite exclusivement des fonctions de recherche textuelle. Il décrit également les fonctions **Distribution**, **Bit**, **Cryptage** et **Information**. Il décrit également en détail l'ensemble de fonctions et de modificateurs pouvant être utilisés avec le modificateur **GROUP BY** avec des exemples appropriés.

OBJECTIFS

Après avoir étudié cette unité, vous devriez être capable de:

- Expliquer et appliquer les fonctions de recherche **Full text**
- Décrire diverses fonctions de **casting**
- Discuter de l'importance des fonctions **Bit**, **Cryptage** et **Information**
- Discuter des fonctions et des modificateurs à utiliser avec les clauses **Group BY**

II. FONCTIONS DE RECHERCHE DE TEXTE INTEGRAL

MySQL prend en charge l'indexation de texte intégral et la recherche:

Un index de texte intégral dans MySQL est un index de type **FULLTEXT**.

Les index de texte intégral ne peuvent être utilisés qu'avec des tables **MyISAM** et ne peuvent être créés que pour des colonnes **CHAR**, **VARCHAR** ou **TEXT**.

Une définition d'index **FULLTEXT** peut être donnée dans l'instruction **CREATE TABLE** lorsqu'une table est créée ou ajoutée ultérieurement à l'aide d'**ALTER TABLE** ou **CREATE INDEX**.

Pour les grands ensembles de données, il est beaucoup plus rapide de charger vos données dans une table sans index FULLTEXT, puis de créer l'index par la suite, plutôt que de charger des données dans une table comportant un index FULLTEXT existant.

La recherche en texte intégral est effectuée à l'aide de la syntaxe MATCH () ... AGAINST. MATCH () prend une liste séparée par des virgules qui nomme les colonnes à rechercher. AGAINST prend une chaîne à rechercher et un modificateur facultatif indiquant le type de recherche à effectuer. La chaîne de recherche doit être une chaîne littérale et non une variable ou un nom de colonne. **Il existe trois types de recherches en texte intégral:**

- **Une recherche booléenne** interprète la chaîne de recherche en utilisant les règles d'un langage de requête spécial. La chaîne contient les mots à rechercher. Elle peut également contenir des opérateurs qui spécifient des exigences telles qu'un mot doit être présent ou absent dans les lignes correspondantes, **ou qu'il doit être pondéré plus haut ou plus bas que la normale.** Les mots courants tels que "certains" ou "alors" sont des mots vides et ne correspondent pas s'ils sont présents dans la chaîne de recherche. Le modificateur IN BOOLEAN MODE spécifie une recherche booléenne.
- **Une recherche en langage naturel** interprète la chaîne de recherche comme une phrase en langage humain naturel (une phrase en texte libre). Il n'y a pas d'opérateurs spéciaux. La liste de mots vides s'applique. En outre, les mots présents dans 50% ou plus des lignes sont considérés comme communs et ne correspondent pas. Les recherches en texte intégral sont des recherches en langage naturel si le modificateur IN NATURAL LANGUAGE MODE est fourni ou si aucun modificateur n'est fourni.
- **Une recherche d'extension de requête** est une modification d'une recherche en langage naturel. La chaîne de recherche est utilisée pour effectuer une recherche en langage naturel. Ensuite, les mots des lignes les plus pertinentes renvoyées par la recherche sont ajoutés à la chaîne de recherche et la recherche est répétée. La requête renvoie les lignes de la deuxième recherche. Le modificateur IN NATURAL LANGUAGE WITH QUERY EXPANSION ou WITH QUERY EXPANSION spécifie une recherche d'extension de requête.

Les modificateurs IN NATURAL LANGUAGE MODE et IN NATURAL LANGUAGE WITH QUERY EXPANSION ont été ajoutés dans MySQL 5.1.7.

II.1. RECHERCHES BOOLEENNES EN TEXTE INTEGRAL

MySQL peut effectuer des recherches booléennes en texte intégral à l'aide du modificateur IN BOOLEAN MODE:

APPLICATION

1ere étape :

Création de la table (nous créerons la table sans l'index **FULLTEXT** car il est très difficile de le faire pendant la création de la table, il est donc alors conseillé de le faire après la création de celle-ci à travers la commande **ALTER TABLE** comme indiquer ci-dessous)

D'où

```
mysql> create table articles (id int primary key auto_increment, titre varchar(30), description varchar(50), prix varchar(20));
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> alter table articles add fulltext (titre);
Query OK, 0 rows affected (0.44 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> alter table articles add fulltext (description);
Query OK, 0 rows affected (0.33 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Après la description, nous aurons :

```
mysql> desc articles;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   | PRI | NULL    | auto_increment |
| titre      | varchar(30)   | YES  | MUL | NULL    |                 |
| description | varchar(50)   | YES  | MUL | NULL    |                 |
| prix       | varchar(20)   | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

2eme étape : insertion des données dans la table.

```
mysql>
mysql> insert into articles values (1,'tutoriel MySQL','GSBD signierait...','200$'),(2,'Comment utiliser MySQL',"apres l'avoir lu...","250$"),(3,'Optimiser MySQL','Ici nous montrerons comm...', '190$'),(4,'1001 les Racourcis...', 'ne jamais raciné MySQL...', '230$'),(5,'Avantages de MySQL','Profiter efficacement..', '310$'),(6,'Securité en MySQL','Si bien configurer...', '270$'),(7,'Transaction MySQL','transferer les donn...', '230$'),(8,'Fonctions MySQL','Manipulation normale...', '325$'),(9,'Langage de definition','La structure MySQL fait...', '395$'),(10,'langage de manipulation','Mettre a jour les donn...', '220$');
Query OK, 10 rows affected (0.07 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

D'où on aura:

```
mysql> select * from articles;
```

```
mysql> select * from articles;
```

id	titre	description	prix
1	tutoriel MySQL	GSBD signierait...	200\$
2	Comment utiliser MySQL	apres l'avoir lu...	250\$
3	Optimiser MySQL	Ici nous montrerons comm....	190\$
4	1001 les Racourcis....	ne jamais raciné MySQL...	230\$
5	Avantages de MySQL	Profiter efficacement..	310\$
6	Securité en MySQL	Si bien configurer...	270\$
7	Transaction MySQL	transférer les donn...	230\$
8	Fonctions MySQL	Manipulation normale...	325\$
9	Langage de definition	La structure MySQL fait...	395\$
10	langage de manipulation	Mettre a jour les donn...	220\$

```
10 rows in set (0.00 sec)
```

3eme étape : la recherche

Pour le faire, c'est très simple. Il suffit juste de bien tenir compte des syntaxe ci-dessous :

Application :

```
mysql> select * from articles Where Match(titre,description) Against ('+mysql-yoursql' in boolean mode)
```

id	titre	description	prix
1	tutoriel MySQL	GSBD signierait...	200\$
2	Comment utiliser MySQL	apres l'avoir lu...	250\$
3	Optimiser MySQL	Ici nous montrerons comm....	190\$
4	1001 les Racourcis....	ne jamais raciné MySQL...	230\$
5	Avantages de MySQL	Profiter efficacement..	310\$
6	Securité en MySQL	Si bien configurer...	270\$
7	Transaction MySQL	transférer les donn...	230\$
8	Fonctions MySQL	Manipulation normale...	325\$
9	Langage de definition	La structure MySQL fait...	395\$

```
9 rows in set (0.32 sec)
```

4eme étape : testons la recherche sans le mot **MySQL** mais avec le mot **langage** dépendant juste de la colonne **titre** :

```
mysql> select * from articles Where Match(titre) Against ('+langage -mysql' in boolean mode);
```

id	titre	description	prix
9	Langage de definition	La structure MySQL fait...	395\$
10	langage de manipulation	Mettre a jour les donn...	220\$

```
2 rows in set (0.00 sec)
```

5eme étape : cette fois-ci, essayons la recherche avec la dépendance des colonnes **titre** et **description**

```
mysql> select * from articles Where Match(titre,description) Against ('+langage -mysql' in boolean mode);
+-----+-----+-----+
| id | titre                | description                | prix |
+-----+-----+-----+
| 10 | langage de manipulation | Mettre a jour les donn... | 220$ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

6eme étape : essayons maintenant sans **langage** et sans **MySQL**

```
mysql> select * from articles Where Match(titre,description) Against ('-langage -mysql' in boolean mode);
Empty set (0.00 sec)
```

7eme étape : avec le mot **langage** sans **MySQL**

```
mysql> select * from articles Where Match(titre,description) Against ('+langage +mysql' in boolean mode);
+-----+-----+-----+
| id | titre                | description                | prix |
+-----+-----+-----+
| 9  | Langage de definition | La structure MySQL fait... | 395$ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Puis les exemples qui suivent :

NB : Il est possible d'entamer la recherche avec juste le mot qui existe (ex : + MySQL), le contraire n'est pas possible.

```
mysql> select * from articles Where Match(titre) Against ('+langage ' in boolean mode);
+-----+-----+-----+
| id | titre                | description                | prix |
+-----+-----+-----+
| 9  | Langage de definition | La structure MySQL fait... | 395$ |
| 10 | langage de manipulation | Mettre a jour les donn... | 220$ |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from articles Where Match(titre,description) Against ('+langage ' in boolean mode);
+-----+-----+-----+
| id | titre                | description                | prix |
+-----+-----+-----+
| 9  | Langage de definition | La structure MySQL fait... | 395$ |
| 10 | langage de manipulation | Mettre a jour les donn... | 220$ |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> select * from articles Where Match(titre) Against ('+mysql ' in boolean mode);
```

id	titre	description	prix
1	tutoriel MySQL	GSD signierait...	200\$
2	Comment utiliser MySQL	apres l'avoir lu...	250\$
3	Optimiser MySQL	Ici nous montrerons comm....	190\$
5	Avantages de MySQL	Profiter efficacement..	310\$
6	Securité en MySQL	Si bien configurer...	270\$
7	Transaction MySQL	transférer les donn...	230\$
8	Fonctions MySQL	Manipulation normale...	325\$

```
7 rows in set (0.00 sec)
```

```
mysql> select * from articles Where Match(titre,description) Against ('+mysql ' in boolean mode);
```

id	titre	description	prix
1	tutoriel MySQL	GSD signierait...	200\$
2	Comment utiliser MySQL	apres l'avoir lu...	250\$
3	Optimiser MySQL	Ici nous montrerons comm....	190\$
4	1001 les Racourcis....	ne jamais raciné MySQL...	230\$
5	Avantages de MySQL	Profiter efficacement..	310\$
6	Securité en MySQL	Si bien configurer...	270\$
7	Transaction MySQL	transférer les donn...	230\$
8	Fonctions MySQL	Manipulation normale...	325\$
9	Langage de definition	La structure MySQL fait...	395\$

```
9 rows in set (0.00 sec)
```

En un mot, la recherche s'est effectuée à travers toutes les lignes dans les deux colonnes (titre et description) c'est adire toute ligne contenant le mot MySQL, a été affiché en dehors du mot YourSQL.

Les opérateurs + et - indiquent qu'un mot doit être **présent** ou **absent**, respectivement, pour qu'une correspondance se produise. Ainsi, cette requête récupère toutes les lignes contenant le mot "MySQL" mais ne contenant pas le mot "YourSQL".

Remarque: pour implémenter cette fonctionnalité, MySQL utilise ce que l'on appelle parfois **la logique booléenne implicite**, dans laquelle:

+ signifie ET, - signifie NOT.

[Pas d'opérateur] impliquant OU

Les recherches booléennes en texte intégral présentent les caractéristiques suivantes:

- ❖ **Ils n'utilisent pas le seuil de 50%.** (pas de recherche à travers le mot contenant une partie du mot en recherche)
- ❖ **Ils ne trient pas automatiquement les lignes par ordre de pertinence décroissante.** Vous pouvez voir ceci dans le résultat de la requête précédente: La ligne avec la pertinence la plus

élevée est celle qui contient deux fois «MySQL», mais elle est répertoriée en dernier, pas en premier.

- ❖ Ils peuvent fonctionner même sans index FULLTEXT, bien qu'une recherche exécutée de cette manière serait assez lente.
- ❖ Les paramètres de texte intégral de longueur de mot minimale et maximale s'appliquent.
- ❖ La liste de mots vides s'applique.
- ❖ La fonction de recherche en texte intégral booléen prend en charge les opérateurs suivants:
 - +: un signe plus indique que ce mot doit être présent dans chaque ligne renvoyée.
 - -: Un premier signe moins indique que ce mot ne doit figurer dans aucune des lignes renvoyées.

REMARQUE:

- ❖ L'opérateur - agit uniquement pour exclure les lignes qui sont par ailleurs mises en correspondance avec d'autres termes de recherche. Ainsi, une recherche en mode booléen qui ne contient que les termes précédés de - renvoie un résultat vide. Il ne renvoie pas «toutes les lignes, sauf celles contenant l'un des termes exclus».
- ❖ (Pas d'opérateur): Par défaut (lorsque ni + ni - n'est spécifié), le mot est facultatif, mais les lignes qui le contiennent sont mieux notées. Cela imite le comportement de MATCH () ... AGAINST () sans le modificateur IN BOOLEAN MODE.
- ❖ ><: Ces deux opérateurs permettent de modifier la contribution d'un mot à la valeur de pertinence attribuée à une ligne. **L'opérateur> augmente la contribution et l'opérateur <la diminue.**
- ❖ (): Les parenthèses regroupent les mots en sous-expressions. Les groupes entre parenthèses peuvent être imbriqués.
- ❖ ~: Un tilde de premier plan joue le rôle d'opérateur de négation, ce qui a pour effet de rendre négatif la contribution du mot à la pertinence de la ligne. Ceci est utile pour marquer des mots «bruits». Une ligne contenant un tel mot a une note inférieure à celle des autres, mais n'est pas totalement exclue, contrairement à l'opérateur -.
- ❖ *: l'astérisque sert d'opérateur de troncature (ou de caractère générique). Contrairement aux autres opérateurs, il convient de l'ajouter au mot à affecter. Les mots correspondent s'ils commencent par le mot précédant l'opérateur *.


```
mysql> select * from articles;
```

id	titre	description	prix
1	tutoriel MySQL	GSBD signierait...	200\$
2	Comment utiliser MySQL	apres l'avoir lu...	250\$
3	Optimiser MySQL	Ici nous montrerons comm....	190\$
4	1001 les Racourcis....	ne jamais raciné MySQL...	230\$
5	Avantages de MySQL	Profiter efficacement..	310\$
6	Securité en MySQL	Si bien configurer...	270\$
7	Transaction MySQL	transferer les donn...	230\$
8	Fonctions MySQL	Manipulation normale...	325\$
9	Langage de definition	La structure MySQL fait...	395\$
10	langage de manipulation	Mettre a jour les donn...	220\$

```
10 rows in set (0.00 sec)
```



```
mysql> select * from articles Where Match(titre,description) Against ('+t* ' in boolean mode);
```

id	titre	description	prix
1	tutoriel MySQL	GSBD signierait...	200\$
7	Transaction MySQL	transferer les donn...	230\$

```
2 rows in set (0.00 sec)
```

Cela affichera la recherche concernant tous les mots commençant par la lettre ou le mot précédant l'astérisque c'est-à-dire t dans notre cas.

Si un mot vide ou un mot trop court est spécifié avec l'opérateur de troncature, il ne sera pas extrait d'une requête booléenne. Par exemple, une recherche sur '+ mot + mot d'arrêt *' renverra probablement moins de lignes qu'une recherche sur '+ mot + mot d'arrêt ', car la requête précédente reste telle quelle et nécessite que le mot d'arrêt * soit présent dans un document. La dernière requête est transformée en + mot.

- ❖ ": Une phrase placée entre guillemets (") ne correspond qu'aux lignes qui contiennent la phrase littéralement, telle qu'elle a été saisie. Le moteur de texte intégral divise la phrase en mots et effectue une recherche dans l'index FULLTEXT pour les mots. Les caractères qui ne sont pas des mots ne doivent pas nécessairement correspondre exactement: la recherche d'une phrase requiert uniquement que les correspondances contiennent exactement les mêmes mots que la phrase et dans le même ordre. **Par exemple, "phrase test" correspond à "test, phrase".**

Si la phrase ne contient aucun mot figurant dans l'index, le résultat est vide. Par exemple, si tous les mots sont soit des mots vides, soit plus courts que la longueur minimale des mots indexés, le résultat est vide.

II.2. RECHERCHES EN TEXTE INTEGRAL AVEC EXTENSION DE REQUETE

La recherche en texte intégral prend en charge le développement de requêtes (et en particulier sa variante «**Développement de requêtes en aveugle**»). Cela est généralement utile lorsqu'une

expression de recherche est trop courte, ce qui signifie souvent que l'utilisateur se fie sur la connaissance implicite que le moteur de recherche de texte intégral manque. **Par exemple, un utilisateur recherchant «base de données» peut en réalité signifier que «MySQL», «Oracle», «DB2» et «RDBMS» sont des expressions qui doivent correspondre à «bases de données» et doivent également être renvoyées.** Ceci est une **connaissance implicite**.

Le développement de requête aveugle (également appelé retour automatique de pertinence) est activé en ajoutant **WITH QUERY EXPANSION** ou **IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION** à la suite de la phrase de recherche. **Cela fonctionne en effectuant la recherche deux fois**, où la phrase de recherche pour la deuxième recherche est la phrase de recherche originale concaténée avec les quelques documents les plus pertinents de la première recherche. Ainsi, si l'un de ces documents contient le mot «bases de données» et le mot «MySQL», la deuxième recherche trouve les documents contenant le mot «MySQL» même s'ils ne contiennent pas le mot «base de données». **L'exemple suivant montre cette différence:**

```
mysql> SELECT * FROM articles
-> WHERE MATCH (title,body)
-> AGAINST ('database' IN NATURAL LANGUAGE MODE);
+-----+-----+-----+
| id | title                                | body                                |
+-----+-----+-----+
| 5 | MySQL vs. YourSQL                   | In the following database comparison ... |
| 1 | MySQL Tutorial                      | DBMS stands for DataBase ...         |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM articles
-> WHERE MATCH (title,body)
-> AGAINST ('database' WITH QUERY EXPANSION);
+-----+-----+-----+
| id | title                                | body                                |
+-----+-----+-----+
| 1 | MySQL Tutorial                      | DBMS stands for DataBase ...         |
| 5 | MySQL vs. YourSQL                   | In the following database comparison ... |
| 3 | Optimizing MySQL                   | In this tutorial we will show ...    |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

REMARQUE:

Etant donné que l'expansion des requêtes à l'aveugle tend à augmenter considérablement le bruit en renvoyant des documents non pertinents, il est judicieux de l'utiliser uniquement lorsqu'une expression de recherche est plutôt courte.

II.3. RESTRICTIONS DE TEXTE INTEGRAL

Les recherches en texte intégral sont prises en charge uniquement pour les tables **MyISAM**.

Les recherches en texte intégral peuvent être utilisées avec la plupart des jeux de caractères multi-octets. L'exception est que pour **Unicode**, le jeu de caractères **utf8** peut être utilisé, mais pas le jeu

de caractères **ucs2**. Cependant, bien que les index FULLTEXT sur les colonnes ucs2 ne puissent pas être utilisés, vous pouvez effectuer des recherches en mode BOOLEAN sur une colonne ucs2 ne disposant pas d'un tel index.

Les langues idéographiques telles que le chinois et le japonais n'ont pas de séparateurs de mots. Par conséquent, l'analyseur FULLTEXT ne peut pas déterminer où les mots commencent et finissent dans ces langages et d'autres.

Bien que l'utilisation de plusieurs jeux de caractères dans une même table soit prise en charge, toutes les colonnes d'un index FULLTEXT doivent utiliser le même jeu de caractères et le même classement.

La liste de colonnes MATCH () doit correspondre exactement à la liste de colonnes de la définition d'index FULLTEXT de la table, sauf si MATCH () est IN MODE BOOLEAN. **Les recherches en mode booléen peuvent être effectuées sur des colonnes non indexées, même si elles sont susceptibles d'être lentes.**

L'argument de AGAINST () doit être une chaîne constante.

II.4. REGLAGE PRECIS DE LA RECHERCHE DE TEXTE INTEGRAL MYSQL

La capacité de recherche en texte intégral de MySQL comporte peu de paramètres réglables par l'utilisateur. Vous pouvez exercer plus de contrôle sur le comportement de recherche en texte intégral si vous disposez d'une distribution source MySQL car certaines modifications nécessitent des modifications du code source.

Notez que la recherche en texte intégral est soigneusement réglée pour une efficacité maximale. La modification du comportement par défaut dans la plupart des cas peut en réalité diminuer l'efficacité. **Ne modifiez pas les sources MySQL sauf si vous savez ce que vous faites.**

La plupart des variables de texte intégral décrites dans cette section doivent être définies au moment du démarrage du serveur. Un redémarrage du serveur est nécessaire pour les changer. Ils ne peuvent pas être modifiés pendant l'exécution du serveur.

Certaines modifications de variables nécessitent la reconstruction des index FULLTEXT dans vos tables. Les instructions pour le faire sont données à la fin de cette section.

Les longueurs minimale et maximale des mots à indexer sont définies par les variables système **ft_min_word_len** et **ft_max_word_len**. La valeur minimale par défaut est de quatre caractères. Le maximum par défaut dépend de la version. Si vous modifiez l'une ou l'autre valeur, vous devez reconstruire vos index FULLTEXT. Par exemple, si vous souhaitez que les mots de trois caractères puissent faire l'objet d'une recherche, vous pouvez définir la variable **ft_min_word_len** en mettant les lignes suivantes dans un fichier d'options:

```
[mysqld]  
ft_min_word_len=3
```

Ensuite, vous devez redémarrer le serveur et reconstruire vos index FULLTEXT. Notez en particulier les remarques concernant myisamchk dans les instructions qui suivent cette liste.

Pour remplacer la liste de mots vides par défaut, définissez la variable système **ft_stopword_file**. La valeur de la variable doit être le chemin du fichier contenant la liste des mots vides ou la chaîne vide pour désactiver le filtrage des mots vides. Après avoir modifié la valeur de cette variable ou le contenu du fichier de mots vides, redémarrez le serveur et reconstruisez vos index FULLTEXT.

La liste de mots vides est de forme libre. En d'autres termes, vous pouvez utiliser n'importe quel caractère non alphanumérique, tel que nouvelle ligne, espace ou virgule, pour séparer les mots vides. Les exceptions sont le caractère de soulignement (`_`) et une seule apostrophe (`'`) qui sont traités comme faisant partie d'un mot. Le jeu de caractères de la liste de mots vides est le jeu de caractères par défaut du serveur.

Le seuil de 50% pour les recherches en langage naturel est déterminé par le système de pondération choisi. Pour le désactiver, recherchez la ligne suivante dans **storage / myisam / ftdefs.h**:

```
#define GWS_IN_USE  
GWS_PROB
```

Changer cette ligne en ce qui suit :

```
#define GWS_IN_USE  
GWS_FREQ
```

Recompilez ensuite MySQL. Il n'est pas nécessaire de reconstruire les index dans ce cas.

Remarque: en apportant cette modification, vous réduisez considérablement la capacité de MySQL à fournir des valeurs de pertinence adéquates pour la fonction MATCH (). Si vous avez réellement besoin de rechercher de tels mots courants, il serait préférable d'utiliser plutôt le mode IN BOOLEAN, qui ne respecte pas le seuil de 50%.

II.5. RECHERCHE DE TEXTE INTEGRAL TODO

Amélioration des performances pour toutes les opérations FULLTEXT.

- Opérateurs de proximité

- Prise en charge des mots «toujours indexés». Celles-ci peuvent être toutes les chaînes que l'utilisateur souhaite traiter comme des mots, tels que «C ++», «AS / 400» ou «TCP / IP».
- Prise en charge de la recherche en texte intégral dans les tables **MERGE**.
- Support pour **UCS-2**.
- Faites en sorte que la liste de mots vides dépende de la langue du jeu de données.
- Racine (dépend de la langue du jeu de données).
- Préparateur d'UDF générique, utilisable par l'utilisateur.
- Rendre le modèle plus flexible (en ajoutant des paramètres ajustables à FULLTEXT dans les instructions CREATE TABLE et ALTER TABLE).

QUESTIONS D'AUTO-EVALUATION

1. Une définition d'index _____ peut être donnée dans l'instruction CREATE TABLE lorsqu'une table est créée ou ajoutée ultérieurement à l'aide d'ALTER TABLE ou CREATE INDEX.

III. FONCTIONS DE CASTING

- ❖ **CAST (type expr AS)**
- ❖ **CONVERT (type, expr)**
- ❖ **CONVERT (expr USING transcodingname)**

Les fonctions CAST () et CONVERT () peuvent être utilisées pour prendre une valeur d'un type et produire une valeur d'un autre type.

La valeur de type peut être l'une des suivantes:

- BINARY
- CHAR
- DATE
- DATETIME
- SIGNED [INTEGER]
- TIME
- UNSIGNED [INTEGER]

CAST () et **CONVERT ()** sont disponibles à partir de MySQL 4.0.2. Le type de conversion **CHAR** est disponible à partir de 4.0.6. Le formulaire **USING** de **CONVERT ()** est disponible à partir de la version 4.1.0.

CAST () et CONVERT (... USING ...) sont une syntaxe SQL standard. La forme non USING de CONVERT () est la syntaxe ODBC.

CONVERT () avec **USING** est utilisé pour convertir les données entre différents jeux de caractères. Dans MySQL, les noms de transcodage sont les mêmes que les noms de jeux de caractères correspondants. Par exemple, cette instruction convertit la chaîne 'abc' dans le jeu de caractères par défaut du serveur en chaîne correspondante dans le jeu de caractères utf8:

- ❖ Les fonctions de conversion sont utiles lorsque vous souhaitez créer une colonne avec un type spécifique dans un fichier.

« Instruction CREATE ... SELECT » :

```
CREATE TABLE new_table SELECT CAST('2000-01-01' AS DATE);
```

APPLICATION :

```
mysql> CREATE table nouvel select cast('2000-01-01' as date);
Query OK, 1 row affected (0.41 sec)
Records: 1  Duplicates: 0  Warnings: 0

mysql> select * from nouvel;
+-----+
| cast('2000-01-01' as date) |
+-----+
| 2000-01-01                  |
+-----+
1 row in set (0.00 sec)
```

- ❖ Les fonctions peuvent également être utiles pour trier les colonnes **ENUM** par ordre lexical. Normalement, le tri des colonnes **ENUM** a lieu à l'aide des valeurs **numériques internes**. En convertissant les valeurs en CHAR, vous obtenez un tri lexical:

```
SELECT enum_col FROM tbl_name ORDER BY CAST(enum_col AS CHAR);
```

APPLICATION :

Créons d'abord une nouvelle table que nous nommerons « **nouvel_1** » et qui comportera une colonne de type **ENUM**.

```
mysql> CREATE table nouvel_1 (id int primary key auto_increment, nom enum('text1','text2','56','text3','34'));
Query OK, 0 rows affected (0.08 sec)

mysql> desc nouvel_1;
+-----+-----+-----+-----+-----+
| Field | Type                                | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+
| id    | int(11)                             | NO   | PRI | NULL    | auto_increment |
| nom   | enum('text1','text2','56','text3','34') | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Etape 2 :

```
mysql> insert into nouvel_1(nom) values ('text3'),('34'),('text2'),('56'),('text1');
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> select * from nouvel_1;
+----+-----+
| id | nom   |
+----+-----+
| 1  | text3 |
| 2  | 34    |
| 3  | text2 |
| 4  | 56    |
| 5  | text1 |
+----+-----+
5 rows in set (0.00 sec)

mysql> select nom from nouvel_1 order by cast(nom as char);
+-----+
| nom   |
+-----+
| 34    |
| 56    |
| text1 |
| text2 |
| text3 |
+-----+
5 rows in set (0.00 sec)
```

CAST (str AS BINARY) est la même chose que BINARY str. CAST (expr AS CHAR) traite l'expression en tant que chaîne avec le jeu de caractères par défaut.

REMARQUE:

- Vous ne devez pas utiliser **CAST ()** pour extraire des données dans différents formats, mais plutôt utiliser des fonctions de chaîne telles que **LEFT ()** ou **EXTRACT ()**.

- Pour convertir une chaîne en valeur numérique, vous n'avez normalement rien à faire. Utilisez simplement la valeur de chaîne pour ainsi dire un nombre:

Application

```
mysql> select 1 + '1';
+-----+
| 1 + '1' |
+-----+
|          2 |
+-----+
1 row in set (0.00 sec)
```

- Si vous utilisez un nombre dans un contexte de chaîne, le nombre sera automatiquement converti en chaîne BINARY.

Application :

```
mysql> select concat ('je suis agée de: ', 20, 'ans');
+-----+
| concat ('je suis agée de: ', 20, 'ans') |
+-----+
| je suis agée de: 20ans |
+-----+
1 row in set (0.24 sec)
```

QUESTIONS D'AUTO-EVALUATION

2. Les fonctions ____ et ____ peuvent être utilisées pour prendre une valeur d'un type et produire une valeur d'un autre type.

IV. AUTRES FONCTIONS

IV.1 FONCTIONS DE BITS

MySQL utilise l'arithmétique BIGINT (64 bits) pour les opérations sur les bits, de sorte que ces opérateurs ont une plage maximale de 64 bits.

- | **BITWISE OU:**

Application :

```
mysql> select 29 | 15;
+-----+
| 29 | 15 |
+-----+
|      31 |
+-----+
1 row in set (0.25 sec)
```

➤ **Le résultat est un entier non signé de 64 bits. & BITWISE ET:**

Application :

```
mysql> select 29 & 15;
+-----+
| 29 & 15 |
+-----+
|      13 |
+-----+
1 row in set (0.00 sec)
```

Le résultat est un entier non signé de 64 bits.

➤ **^ XOR bit à bit:**

Application :

```
mysql> select 1 ^ 1;
+-----+
| 1 ^ 1 |
+-----+
|      0 |
+-----+
1 row in set (0.00 sec)

mysql> select 1 ^ 0;
+-----+
| 1 ^ 0 |
+-----+
|      1 |
+-----+
1 row in set (0.00 sec)

mysql> select 11 ^ 3;
+-----+
| 11 ^ 3 |
+-----+
|      8 |
+-----+
1 row in set (0.00 sec)
```

Le résultat est un entier non signé de 64 bits.

➤ **<<SHIFTL ou décalé le numéro à gauche :**

Application :

```
mysql> select 1<<2;
+-----+
| 1<<2 |
+-----+
|    4 |
+-----+
1 row in set (0.00 sec)
```

Le résultat est un entier non signé de 64 bits.

➤ **>>: Décale le numéro long (BIGINT) vers la droite.**

Application :

```
mysql> select 4>>2;
+-----+
| 4>>2 |
+-----+
|    1 |
+-----+
1 row in set (0.00 sec)
```

Le résultat est un entier non signé de 64 bits.

➤ **~: Inverser tous les bits.**

Application :

```
mysql> select 5 & ~1;
+-----+
| 5 & ~1 |
+-----+
|      4 |
+-----+
1 row in set (0.00 sec)

mysql> select 6 & ~1;
+-----+
| 6 & ~1 |
+-----+
|      6 |
+-----+
1 row in set (0.00 sec)
```

Le résultat est un entier non signé de 64 bits.

➤ **BIT_COUNT (N): Retourne le nombre de bits définis dans l'argument N.**

Application :

```
mysql> select bit_count(29);
+-----+
| bit_count(29) |
+-----+
|              4 |
+-----+
1 row in set (0.24 sec)

mysql> select bit_count(30);
+-----+
| bit_count(30) |
+-----+
|              4 |
+-----+
1 row in set (0.00 sec)

mysql> select bit_count(100);
+-----+
| bit_count(100) |
+-----+
|              3 |
+-----+
1 row in set (0.00 sec)
```

IV.2. FONCTIONS DE CHIFFREMENT

❖ AES_ENCRYPT (str, key_str)

❖ AES_DECRYPT (str, key_str)

Ces fonctions permettent **le cryptage / décryptage** des données à l'aide de l'algorithme officiel **AES (Advanced Encryption Standard)**, précédemment appelé **RIJNDAEL**. Un codage avec une longueur de clé de 128 bits est utilisé, mais vous pouvez l'étendre jusqu'à 256 bits en modifiant la source. Nous avons choisi 128 bits parce que c'est beaucoup plus rapide et généralement assez sécurisé.

Les arguments d'entrée peuvent avoir n'importe quelle longueur. Si l'un des arguments est NULL, le résultat de cette fonction est également NULL.

Comme AES est un algorithme de niveau bloc, le remplissage est utilisé **pour coder des chaînes de longueur inégale**. La longueur de chaîne de résultat peut donc être calculée sous la forme $16 * (\text{trunc}(\text{chaîne_longue} / 16) + 1)$.

Si AES_DECRYPT () détecte des données non valides ou un remplissage incorrect, il renvoie NULL.

Toutefois, il est possible pour AES_DECRYPT () de renvoyer une valeur non NULL (éventuellement une erreur) si les données d'entrée ou la clé sont non valides.

Vous pouvez utiliser les fonctions AES pour stocker des données sous une forme cryptée en modifiant vos requêtes:

APPLICATION :

1. LA LONGUEUR DE LA CHAINE DE RESULTAT :

Lorsque nous cryptons une expression, celle-ci devient illisible c'est-à-dire qu'elle se transforme en une expression incompréhensible par l'humain. Cette expression cryptée, a une longueur déterminable à travers la formule ci-dessous :

$$16 * (\text{trunc}(\text{longueur_du_mot} / 16) + 1)$$

Comment cela se calcule vraiment ?

Supposons que nous avons eu à créer une table **t** comportant deux (2) colonnes **text** et **password**.

Créons la table **t** :

```
mysql> create table t (text varchar(50),password varchar(50));
Query OK, 0 rows affected (0.31 sec)
```

Retenons que nos deux colonnes sont en **varchar**, et colonne qui doit subir les transformations de texte ou les cryptages est bien sûr la colonne **password** et elle est en **varchar (50)**.

Commençons alors les calculs :

Soit la formule : $16 * (\text{trunc}(\text{longueur_du_mot} / 16) + 1)$

- ⇒ Avec la longueur du mot étant 50, nous aurons donc $16 * (\text{trunc}(50/16) + 1)$
- ⇒ $16 * (3,125 + 1)$
- ⇒ $16 * 4,125$
- ⇒ 66 en minimum, environ **varbinary(100)**.

Mais pourquoi cela donnerait-il un varbinary au lieu du varchar ?

Comme on a su le dire ci-dessus, après cryptage d'une expression celle-ci se transforme en une expression incompréhensible qui est en réalité une expression en varbinary. Nous retenons donc que pour crypter une expression, celle-ci doit être en **varchar** afin d'être transformé en **varbinary**.

2. COMMENT CRYPTER ?

Crypter une expression est une chose très facile à faire, pourquoi ?

Parce qu'il est juste question de prendre en référence ou en considération trois (3) éléments :

- ❖ AES_ENCRYPT (utilisé pour le cryptage)
- ❖ Le mot à crypter et
- ❖ La clé de cryptage (qui ne doit être oubliée).

Soit la table ci-dessous :

```
mysql> desc t;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| text       | varchar(50)   | YES  |     | NULL    |       |
| password   | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Insérons maintenant :

```
mysql> insert into t (text,password) values ('1',aes_encrypt('albert2008','la_cle')),
-> ('2',aes_encrypt('12/06/1991','la_cle'));
Query OK, 2 rows affected (0.35 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Retenons qu'ici nous avons eu à utiliser la clé qui s'appelle **la_cle**. Pour les raisons de sécurité, il est demandé de jamais oublier la clé sinon il nous sera impossible de décrypter les expressions cryptées.

Vous pouvez obtenir encore plus de sécurité en ne transférant pas la clé sur la connexion pour chaque requête. Pour ce faire, **vous pouvez la stocker dans une variable côté serveur au moment de la connexion.**

Exemple:

```
mysql> select @password:='mot de passe';
+-----+
| @password:='mot de passe' |
+-----+
| mot de passe              |
+-----+
1 row in set (0.00 sec)
```

Insérons pour voir le résultat :

```
mysql> insert into t values (3,aes_encrypt('bonjour',@password));
Query OK, 1 row affected (0.00 sec)
```

NB : Pour la 3eme ligne, nous l'avons encrypté avec la clé incorporée dans la variable **@password**.

DECRYPTAGE

```
mysql> select AES_DECRYPT(password,'la_cle') from t where text=1 or text=2;
+-----+
| AES_DECRYPT(password,'la_cle') |
+-----+
| albert2008                     |
| 12/06/1991                    |
+-----+
2 rows in set (0.31 sec)
```

```
mysql> select AES_DECRYPT(password,'mot de passe') from t where text=3;
+-----+
| AES_DECRYPT(password,'mot de passe') |
+-----+
| bonjour                             |
+-----+
1 row in set (0.03 sec)
```

CRYPTAGE AVEC DES :

AES_ENCRYPT () et **AES_DECRYPT ()** ont été ajoutés à **MySQL 4.0.2** et peuvent être considérés comme les fonctions de cryptage les plus sécurisées sur le plan cryptographique actuellement disponibles dans MySQL.

DECODE (crypt_str, pass_str): décrypte la chaîne cryptée **crypt_str** en utilisant **pass_str** comme mot de passe. **crypt_str** doit être une chaîne renvoyée par **ENCODE ()**.

ENCODE (str, pass_str): Cryptez str en utilisant pass_str comme mot de passe. Pour déchiffrer le résultat, utilisez DECODE (). Le résultat est une chaîne binaire de la même longueur que la chaîne. Si vous souhaitez l'enregistrer dans une colonne, utilisez un type de colonne BLOB.

DES_DECRYPT (str_to_decrypt [, key_str]): déchiffre une chaîne chiffrée avec DES_ENCRYPT (). En cas d'erreur, cette fonction renvoie NULL. Notez que cette fonction ne fonctionne que si MySQL a été configuré avec le support SSL.

- Si aucun argument **key_str** n'est donné, **DES_DECRYPT ()** examine le premier octet de la chaîne chiffrée pour déterminer le numéro de clé DES utilisé pour chiffrer la chaîne d'origine, puis lit la clé à partir du fichier de clé DES pour décrypter le message. Pour que cela fonctionne, l'utilisateur doit disposer du privilège SUPER. Le fichier de clé peut être spécifié avec l'option de serveur --des-key-file.
- Si vous transmettez à cette fonction un argument **key_str**, cette chaîne est utilisée comme clé de déchiffrement du message.
- Si l'argument **str_to_decrypt** ne ressemble pas à une chaîne chiffrée, MySQL renverra le str_to_decrypt donné.

`DES_ENCRYPT(str_to_encrypt[, (key_num | key_str)])`

Crypte la chaîne avec la clé donnée en utilisant l'algorithme Triple-DES. En cas d'erreur, cette fonction renvoie **NULL**.

APPLICATION :

a. Sans clé

```
mysql> insert into t (text,password) values ('4',des_encrypt('Bonsoir'));
Query OK, 1 row affected (0.00 sec)
```

b. Avec clé

```
mysql> insert into t (text,password) values ('5',des_encrypt('Bonsoir','la_cle'));
Query OK, 1 row affected (0.00 sec)
```

DECRYPTAGE DE DES :

Soit :

```
mysql> select * from t;
+-----+-----+
| text | password |
+-----+-----+
| 1    | i1-3$5±:ÇLc |
| 2    | Z e;5Yü%qD}Ä.\ |
| 3    | t:Eo C0ãw B |
| 4    | ÇfT"´· @ |
| 5    | ð O-qüý |
| 6    | Bravo a vous! |
+-----+-----+
6 rows in set (0.00 sec)
```

a. Sans clé :

```
mysql> select DES_DECRYPT(password) from t where text=4;
+-----+
| DES_DECRYPT(password) |
+-----+
| Bonsoir |
+-----+
1 row in set (0.04 sec)
```

b. Avec clé :

```
mysql> select DES_DECRYPT(password,'la_cle') from t where text=5;
+-----+
| DES_DECRYPT(password,'la_cle') |
+-----+
| Bonsoir |
+-----+
1 row in set (0.17 sec)
```

NB :

- Notez que cette fonction ne fonctionne que si MySQL a été configuré avec le support SSL.
- Le Cryptage sans ne se fait qu'avec le système **DES_ENCRYPT** mais pas avec **AES_ENCRYPT**.

La clé de chiffrement à utiliser est choisie en fonction du deuxième argument de **DES_ENCRYPT** (), s'il en existe un:

CRYPTAGE AVEC ENCRYPT (str [, salt])

Cryptez str en utilisant l'appel système Unix crypt (). L'argument salt devrait être une chaîne avec deux caractères. (Depuis MySQL 3.22.16, salt peut contenir plus de deux caractères.)


```
mysql> SELECT ENCRYPT('hello');  
-> 'VxuFAJXVARROc'
```

ENCRYPT () ignore tous les caractères sauf les 8 premiers caractères de **str**, du moins sur certains systèmes. Ce comportement est déterminé par la mise en œuvre de l'appel système sous-jacent crypt (). **Si crypt () n'est pas disponible sur votre système, ENCRYPT () renvoie toujours NULL.** C'est pourquoi nous vous recommandons d'utiliser plutôt MD5 () ou SHA1 (), car ces deux fonctions existent sur toutes les plateformes.

MD5 (str)

Calcule une somme de contrôle MD5 128 bits pour la chaîne. La valeur est renvoyée sous forme de chaîne de 32 chiffres hexadécimaux, ou NULL si l'argument était NULL. La valeur de retour peut, par exemple, être utilisée comme clé de hachage.

```
mysql> SELECT MD5('testing');  
-> 'ae2b1fca515949e5d54fb22b8ed95575'
```

Il s'agit de "l'algorithme **MD5** Message-Digest de RSA Data Security, Inc."

Application :

a. Sélection ordinaire :

```
mysql> select md5('salut');  
+-----+  
| md5('salut') |  
+-----+  
| 3ed7dceaf266cafef032b9d5db224717 |  
+-----+  
1 row in set (0.00 sec)
```

b. Sélection depuis la table :

```
mysql> insert into t values ('6','Bravo a vous!');
Query OK, 1 row affected (0.00 sec)

mysql> select * from t;
+-----+-----+
| text | password |
+-----+-----+
| 1    | i+@-■3$ r5±:ÇLc-|
| 2    | Z e;5 r@Yü%qD}Ä.\ |
| 3    | t:Eo C0äw ß@
| 4    | ÇfT"´·@ @
| 5    | ð oLqüý@
| 6    | Bravo a vous!
+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> select md5(password) from t where text=5;
+-----+
| md5(password) |
+-----+
| 29895389c278f146d3db76e3abc8ffe5 |
+-----+
1 row in set (0.00 sec)
```

LE CRYPTAGE PASSWORD

PASSWORD (str), OLD_PASSWORD (str)

Calcule et retourne une chaîne de mot de passe à partir du mot de passe en texte brut **str**, ou **NULL** si l'argument était **NULL**. Cette fonction est utilisée pour chiffrer les mots de passe MySQL pour le stockage dans la colonne Mot de passe de la table des droits des utilisateurs.

```
mysql> SELECT PASSWORD('badpwd');
-> '7f84554057dd964b'
```

Le cryptage **PASSWORD ()** est unidirectionnel (**non réversible**).

APPLICATION :

- Sélection ordinaire

```
mysql> select PASSWORD('salut');
+-----+
| PASSWORD('salut') |
+-----+
| *7B272372A6EE59392EA2FAAEB66D8968560807CB |
+-----+
1 row in set (0.01 sec)
```

b. Sélection depuis de la table :

```
mysql> select PASSWORD('salut');
+-----+
| PASSWORD('salut') |
+-----+
| *7B272372A6EE59392EA2FAAEB66D8968560807CB |
+-----+
1 row in set (0.01 sec)
```

PASSWORD () n'effectue pas le cryptage de mot de passe de la même manière que les mots de passe Unix sont cryptés.

REMARQUE:

- ❖ La fonction **PASSWORD ()** est utilisée par le système d'authentification de MySQL Server. Vous ne devez pas l'utiliser dans vos propres applications.
- ❖ Pour cela, utilisez **MD5 ()** ou **SHA1 ()** à la place. Consultez également la RFC 2195 pour plus d'informations sur la gestion sécurisée des mots de passe et de l'authentification dans votre application.

CRYPTAGE AVEC SHA1 (str)

SHA (str) Calcule une somme de contrôle **SHA1** de 160 bits pour la chaîne, comme décrit dans la **RFC 3174 (algorithme de hachage sécurisé)**. La valeur est renvoyée sous forme de chaîne de 40 chiffres hexadécimaux, ou NULL si l'argument était NULL. L'une des utilisations possibles de cette fonction est la clé de hachage. **Vous pouvez également l'utiliser en tant que fonction sécurisée par cryptographie pour stocker les mots de passe.**

```
mysql> SELECT SHA1('abc');
-> 'a9993e364706816aba3e25717850c26c9cd0d89d'
```

a. Sélection ordinaire :

```
mysql> select SHA1('salut');
+-----+
| SHA1('salut') |
+-----+
| 1bfbd3f35b1359fc6b6f93893874cf23a50293de5 |
+-----+
1 row in set (0.00 sec)
```

b. Sélection depuis la table :

```
mysql> select SHA1(password) from t where text=6;
+-----+
| SHA1(password) |
+-----+
| 17694534835b2a85082ff008f19ac1b7fa2e28a4 |
+-----+
1 row in set (0.00 sec)
```

SHA1 () a été ajouté à **MySQL 4.0.2** et peut être considéré comme un équivalent plus sûr du point de vue cryptographique de **MD5 ()**. **SHA ()** est synonyme de **SHA1 ()**.

IV.3. FONCTIONS D'INFORMATION

FONCTION BENCHMARK (compter, expr)

La fonction **BENCHMARK ()** exécute l'expression **expr** plusieurs fois. Il peut être utilisé pour indiquer la rapidité avec laquelle **MySQL** traite l'expression. La valeur de résultat est toujours **zéro 0**. L'utilisation prévue provient du client **MySQL**, qui indique les temps d'exécution des requêtes:

```
mysql> select BENCHMARK(1000000,ENCODE('helo','goodbye'));
+-----+
| BENCHMARK(1000000,ENCODE('helo','goodbye')) |
+-----+
| 0 |
+-----+
1 row in set (0.47 sec)

mysql> select BENCHMARK(1000000,ENCODE('helo','goodbye'));
+-----+
| BENCHMARK(1000000,ENCODE('helo','goodbye')) |
+-----+
| 0 |
+-----+
1 row in set (0.15 sec)

mysql> select BENCHMARK(1000000,ENCODE('helo','goodbye'));
+-----+
| BENCHMARK(1000000,ENCODE('helo','goodbye')) |
+-----+
| 0 |
+-----+
1 row in set (0.15 sec)

mysql> select BENCHMARK(1000000,ENCODE('helo','goodbye'));
+-----+
| BENCHMARK(1000000,ENCODE('helo','goodbye')) |
+-----+
| 0 |
+-----+
1 row in set (0.14 sec)
```

Le temps indiqué est le temps écoulé côté **client**, et non pas le temps **CPU** du côté serveur. Il peut être judicieux d'exécuter BENCHMARK () plusieurs fois et d'interpréter le résultat en fonction de l'importance de la charge de la machine serveur.

FONCTION CHARSET (str)

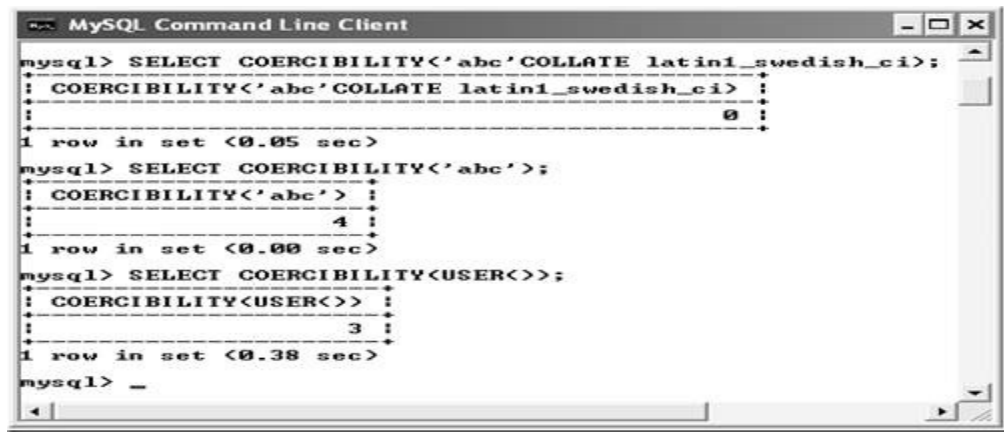
Renvoie le jeu de caractères de l'argument de chaîne.

```
mysql> select CHARSET(_utf8'abc');
+-----+
| CHARSET(_utf8'abc') |
+-----+
| utf8 |
+-----+
1 row in set (0.01 sec)
```

CHARSET () a été ajouté à **MySQL 4.1.0**.

FONCTION COERCIBILITÉ (str)

Renvoie la valeur de coercibilité du classement de l'argument de chaîne.



```
mysql> SELECT COERCIBILITY<'abc' COLLATE latin1_swedish_ci>;
+-----+
| COERCIBILITY<'abc' COLLATE latin1_swedish_ci> |
+-----+
| 0 |
+-----+
1 row in set (0.05 sec)

mysql> SELECT COERCIBILITY<'abc'>;
+-----+
| COERCIBILITY<'abc'> |
+-----+
| 4 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT COERCIBILITY<USER>;
+-----+
| COERCIBILITY<USER> |
+-----+
| 3 |
+-----+
1 row in set (0.38 sec)

mysql> _
```

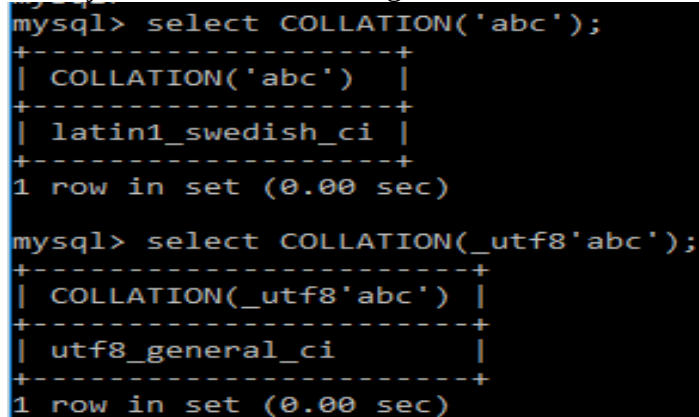
Les valeurs de retour ont les significations suivantes:

Coercibility	Meaning
0	Explicit collation
1	No collation
2	Implicit collation
3	Coercible

Les valeurs inférieures ont une priorité plus élevée. **COERCIBILITY ()** a été ajouté dans **MySQL 4.1.1**.

FONCTION COLLATION (str)

Renvoie le classement du jeu de caractères de l'argument de chaîne.



```
mysql> select COLLATION('abc');
+-----+
| COLLATION('abc') |
+-----+
| latin1_swedish_ci |
+-----+
1 row in set (0.00 sec)

mysql> select COLLATION(_utf8'abc');
+-----+
| COLLATION(_utf8'abc') |
+-----+
| utf8_general_ci |
+-----+
1 row in set (0.00 sec)
```

COLLATION () a été ajouté à **MySQL 4.1.0**.

FONCTION CONNECTION_ID ()

Renvoie l'**ID** de connexion (**ID de thread**) de la connexion. Chaque connexion a son propre identifiant unique.

```
mysql> select CONNECTION_ID();
+-----+
| CONNECTION_ID() |
+-----+
|                1 |
+-----+
1 row in set (0.03 sec)
```

FONCTION CURRENT USER ()

Renvoie le nom d'utilisateur et le nom d'hôte avec lesquels la session en cours a été authentifiée.
 Cette valeur correspond au compte utilisé pour évaluer vos privilèges d'accès. Il peut être différent de la valeur de USER ().

```
mysql> SELECT USER();
```

USER()
root@localhost

```
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM MYSQL.USER;
```

Host	User	Password		Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv	Drop_priv	Reload_priv	Shutdown_priv	Process_priv	File_priv	Grant_priv	References_priv	Index_priv	Alter_priv	Show_db_priv	Super_priv	Create_tmp_table_priv	Lock_tables_priv	Execute_priv	Repl_slave_priv	Repl_client_priv	Create_view_priv	Show_view_priv	Create_routine_priv	Alter_routine_priv	Create_user_priv	Event_priv	Trigger_priv	Ssl_type	Ssl_cipher	X509_issuer	X509_subject	max_questions	max_updates	max_connections	max_user_connections			
localhost	root	*1042C939D306FC7519F450239963393F652B68CE	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	
127.0.0.1	root		Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	
localhost			N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N

3 rows in set (0.00 sec)

```
mysql> SELECT CURRENT_USER();
+-----+
| CURRENT_USER() |
+-----+
| root@localhost |
+-----+
1 row in set (0.00 sec)
```

FONCTION DATABASE () :

Renvoie le nom de la base de données (actuel) par défaut.

```
mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| ensp       |
+-----+
1 row in set (0.00 sec)
```

S'il n'y a pas de base de données par **défaut**, **DATABASE ()** renvoie **NULL** à partir de **MySQL 4.1.1** et la chaîne vide avant celle-ci.

FONCTION SQL_CALC_FOUND_ROWS ()

Le second **SELECT** retournera un nombre indiquant le nombre de lignes que le premier **SELECT** aurait renvoyé s'il avait été écrit sans la clause **LIMIT**. (Si l'instruction **SELECT** précédente n'inclut pas l'option **SQL_CALC_FOUND_ROWS**, **FOUND_ROWS ()** peut renvoyer un résultat différent lorsque **LIMIT** est utilisé ou non.).

```
mysql> select SQL_CALC_FOUND_ROWS * FROM t LIMIT 3;
+-----+-----+
| text | password |
+-----+-----+
| 1    | i1[0] 3$ r5±:Ç llc|| |
| 2    | Z e;5 r[0]YÜ%qD}Ä.\ |
| 3    | t:Eo | C0ăw B[0] |
+-----+-----+
3 rows in set (0.02 sec)

mysql> SELECT FOUND_ROWS();
+-----+
| FOUND_ROWS() |
+-----+
| 6             |
+-----+
1 row in set (0.00 sec)
```

REMARQUE: Si vous utilisez **SELECT SQL_CALC_FOUND_ROWS**, **MySQL** doit calculer le nombre de lignes dans le jeu de résultats complet. Cependant, cela est plus rapide que de **ré exécuter** la requête sans **LIMIT**, car le jeu de résultats n'a pas besoin d'être envoyé au client.

SQL_CALC_FOUND_ROWS et **FOUND_ROWS ()** peuvent être utiles dans les situations où vous souhaitez limiter le nombre de lignes renvoyées par une requête, mais également déterminer le nombre de lignes du jeu de résultats complet sans l'exécuter à nouveau. Un exemple est un script Web qui présente un affichage paginé contenant des liens vers les pages montrant d'autres sections d'un résultat de recherche. L'utilisation de **FOUND_ROWS ()** vous permet de déterminer combien d'autres pages sont nécessaires pour le reste du résultat.

FONCTION LAST_INSERT_ID ()

LAST_INSERT_ID (expr)

Retourne la dernière valeur générée automatiquement qui a été insérée dans une colonne **AUTO_INCREMENT**.

```
mysql> SELECT LAST_INSERT_ID();
+-----+
| LAST_INSERT_ID() |
+-----+
|                0 |
+-----+
1 row in set (0.00 sec)
```

Le dernier ID généré est conservé sur le serveur connexion par connexion. Cela signifie que la valeur renvoyée par la fonction à un client donné est la valeur **AUTO_INCREMENT** la plus récente générée par ce client. La valeur ne peut pas être affectée par d'autres clients, même s'ils génèrent leurs propres valeurs **AUTO_INCREMENT**. Ce comportement garantit que vous pouvez récupérer votre propre ID sans vous préoccuper de l'activité des autres clients et sans avoir besoin de verrous ou de transactions.

- ❖ La valeur de **LAST_INSERT_ID ()** n'est pas modifiée si vous mettez à jour la colonne **AUTO_INCREMENT** d'une ligne avec une valeur non magique (c'est-à-dire, une valeur autre que **NULL** et non 0). Si vous insérez plusieurs lignes en même temps avec une instruction **insert**, **LAST_INSERT_ID ()** renvoie la valeur de la première ligne insérée. La raison en est qu'il est possible de reproduire facilement la même instruction **INSERT** sur un autre serveur.
- ❖ Si **expr** est donné en argument à **LAST_INSERT_ID ()**, alors la valeur de l'argument est renvoyée par la fonction et définie comme la prochaine valeur à renvoyer par **LAST_INSERT_ID ()**. Ceci peut être utilisé pour simuler des séquences:



```
MySQL Command Line Client
mysql> CREATE TABLE sequence(id INT NOT NULL);
Query OK, 0 rows affected (0.50 sec)

mysql> INSERT INTO sequence VALUES(0);
Query OK, 1 row affected (0.06 sec)

mysql> UPDATE sequence SET id = LAST_INSERT_ID(id+1);
Query OK, 1 row affected (0.41 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT LAST_INSERT_ID();
+-----+
| LAST_INSERT_ID() |
+-----+
|                1 |
+-----+
1 row in set (0.00 sec)

mysql> _
```

- ❖ L'instruction **UPDATE** incrémente le compteur de séquence et provoque l'appel suivant à **LAST_INSERT_ID ()** pour renvoyer la valeur mise à jour. L'instruction **SELECT** récupère cette valeur.

FONCTION VERSION ()

Retourne une chaîne qui indique la version du serveur **MySQL**.

```
mysql> SELECT VERSION();
+-----+
| VERSION() |
+-----+
| 5.1.53-community-log |
+-----+
1 row in set (0.00 sec)
```

REMARQUE: Si votre chaîne de version se termine par -log, cela signifie que la journalisation est activée.

IV.4 FONCTIONS DIVERSES

FONCTION FORMAT (X, D)

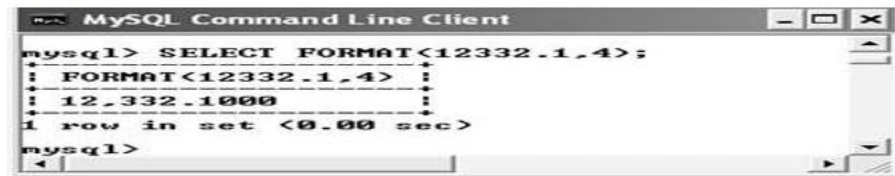
Formate le nombre **X** en un format du type '#, ###, ###. ##', **Arrondi** à **D** décimales et renvoie le résultat sous forme de chaîne. Si **D** est égal à 0, le résultat n'aura pas de point décimal ni de fraction.

Exemple 1:



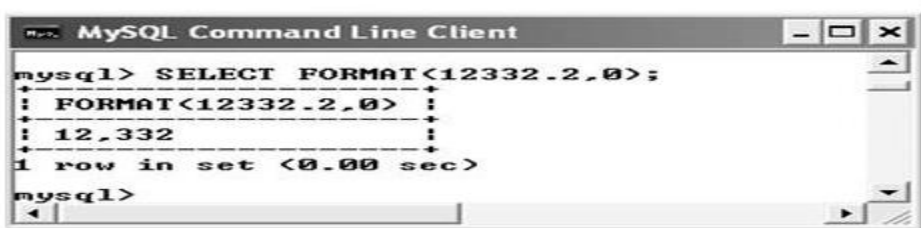
```
mysql> SELECT FORMAT(12332.123456,4);
+-----+
| FORMAT(12332.123456,4) |
+-----+
| 12,332.1235 |
+-----+
1 row in set (0.06 sec)
mysql> _
```

Exemple ♦ 2:



```
mysql> SELECT FORMAT(12332.1,4);
+-----+
| FORMAT(12332.1,4) |
+-----+
| 12,332.1000 |
+-----+
1 row in set (0.00 sec)
mysql>
```

Exemple ♦ 3:



```
mysql> SELECT FORMAT(12332.2,0);
+-----+
| FORMAT(12332.2,0) |
+-----+
| 12,332 |
+-----+
1 row in set (0.00 sec)
mysql>
```

QUESTIONS D'AUTO-EVALUATION

3. La sortie de l'opération **AND** au niveau des bits **29** et **15** suivante est _____.
4. Il est possible pour **AES_DECRYPT ()** de renvoyer un _____ (éventuellement un déchet) si les données d'entrée ou la clé sont invalides.

V. FONCTIONS ET MODIFICATEURS A UTILISER AVEC LES CLAUSES GROUP BY

V.1. FONCTIONS GROUP BY (AGRÉGAT)

Cette section décrit les fonctions de groupe (agrégat) qui agissent sur des ensembles de valeurs. Sauf indication contraire, les fonctions de groupe ignorent les valeurs **NULL**.

Si vous utilisez une fonction group dans une instruction ne contenant aucune clause **GROUP BY**, cela équivaut à un regroupement sur toutes les lignes.

REMARQUE :

- ❖ Les fonctions d'agrégation **SUM ()** et **AVG ()** ne fonctionnent pas avec les valeurs temporelles. (Ils convertissent les valeurs en nombres, ce qui perd la partie après le premier caractère non numérique.).
- ❖ Pour contourner ce problème, vous pouvez convertir en unités numériques, effectuer l'opération d'agrégation et reconvertir en une valeur temporelle.

Exemples:

Soit...

```
mysql> CREATE TABLE T1 (ID INT UNIQUE AUTO_INCREMENT, TEMPS TIME, DDN DATE);
Query OK, 0 rows affected (11.47 sec)

mysql> INSERT INTO T1(TEMPS, DDN) VALUES ('12:23','2015-02-23'),('03:34','2019-05-11'),('20:48','2002-08-22'),('13:34','2009-10-28'),('21:08','2011-12-25');
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM T1;
+----+-----+-----+
| ID | TEMPS | DDN   |
+----+-----+-----+
| 1  | 12:23:00 | 2015-02-23 |
| 2  | 03:34:00 | 2019-05-11 |
| 3  | 20:48:00 | 2002-08-22 |
| 4  | 13:34:00 | 2009-10-28 |
| 5  | 21:08:00 | 2011-12-25 |
+----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT SEC_TO_TIME(SUM(TIME_TO_SEC(TEMPS))) FROM T1;
+-----+
| SEC_TO_TIME(SUM(TIME_TO_SEC(TEMPS))) |
+-----+
| 71:27:00 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT SUM(TO_DAYS(DDN)) FROM T1;
+-----+
| SUM(TO_DAYS(DDN)) |
+-----+
| 3673955 |
+-----+
1 row in set (0.00 sec)
```

1. `SELECT SEC_TO_TIME(SUM(TIME_TO_SEC(time_col))) FROM tbl_name;`
2. `SELECT FROM_DAYS(SUM(TO_DAYS(date_col))) FROM tbl_name;`

FONCTION AVG ([DISTINCT] expr)

Retourne la valeur moyenne d'expr. L'option **DISTINCT** peut être utilisée à partir de **MySQL 5.0.3** pour renvoyer la moyenne des valeurs distinctes d'expr.

AVG () renvoie **NULL** s'il n'y a pas de lignes correspondantes.

Soit....

```
mysql> CREATE TABLE NOTES_ETUDIANT (ID INT UNIQUE AUTO_INCREMENT, NOM VARCHAR(40), NOTES INT(5));
Query OK, 0 rows affected (0.34 sec)
```

```
mysql> SELECT * FROM NOTES_ETUDIANT;
+----+-----+-----+
| ID | NOM           | NOTES |
+----+-----+-----+
| 1  | NGOMA PIERRE  | 16    |
| 2  | MANDZOU SILVAIN | 13    |
| 3  | OBOA GEORGE   | 14    |
| 4  | YOULOU FRANCOIS | 15    |
| 5  | KIWOUZOU ALAIN | 12    |
| 6  | NGOMA PIERRE  | 13    |
| 7  | MANDZOU SILVAIN | 16    |
| 8  | OBOA GEORGE   | 11    |
| 9  | YOULOU FRANCOIS | 13    |
| 10 | KIWOUZOU ALAIN | 10    |
+----+-----+-----+
10 rows in set (0.00 sec)

mysql> SELECT NOM, AVG(NOTES) FROM NOTES_ETUDIANT GROUP BY NOM;
+-----+-----+
| NOM           | AVG(NOTES) |
+-----+-----+
| KIWOUZOU ALAIN | 11.0000    |
| MANDZOU SILVAIN | 14.5000    |
| NGOMA PIERRE   | 14.5000    |
| OBOA GEORGE    | 12.5000    |
| YOULOU FRANCOIS | 14.0000    |
+-----+-----+
5 rows in set (0.28 sec)
```

```
mysql> SELECT student_name, AVG(test_score)
-> FROM student
-> GROUP BY student_name;
```

FONCTION BIT_AND (expr)

Renvoie le bit **AND** et tous les bits d'expr. Le calcul est effectué avec une précision de 64 bits (**BIGINT**).

Cette fonction renvoie **18446744073709551615** s'il n'y a pas de lignes correspondantes. (Il s'agit de la valeur d'une valeur BIGINT non signée avec tous les bits définis sur 1.)

FONCTION BIT_OR (expr)

Renvoie le **OU** au niveau du bit de tous les bits dans expr. Le calcul est effectué avec une précision de 64 bits (**BIGINT**).

Cette fonction renvoie **0** s'il n'y a pas de lignes correspondantes.

FONCTION BIT_XOR (expr)

Renvoie le **XOR** au niveau du bit de tous les bits dans expr. Le calcul est effectué avec une précision de 64 bits (**BIGINT**).

Cette fonction renvoie **0** s'il n'y a pas de lignes correspondantes.

FONCTION COUNT (expr)

Retourne le nombre de valeurs non **NULL** dans les lignes extraites par une instruction **SELECT**.

Exemple:

```
mysql> SELECT student.student_name, COUNT(*)
-> FROM student, course
-> WHERE student.student_id=course.student_id
-> GROUP BY student_name;
```

```
mysql> SELECT NOM, COUNT(*) FROM NOTES_ETUDIANT GROUP BY NOM;
+-----+-----+
| NOM          | COUNT(*) |
+-----+-----+
| KIWOZOU ALAIN | 2        |
| MANDZOU SILVAIN | 2        |
| NGOMA PIERRE  | 2        |
| OBOA GEORGE   | 2        |
| YOLOU FRANCOIS | 2        |
+-----+-----+
5 rows in set (0.01 sec)
```

COUNT (*) diffère quelque peu en ce sens qu'il renvoie le nombre de lignes extraites, qu'elles contiennent ou non des valeurs **NULL**.

FONCTION COUNT (DISTINCT expr, [expr ...])

Retourne le nombre de valeurs différentes non NULL.

Exemple:

```
mysql> SELECT COUNT (DISTINCT results) FROM student;
```

```
mysql> SELECT COUNT(DISTINCT NOTES) FROM NOTES_ETUDIANT;
+-----+
| COUNT(DISTINCT NOTES) |
+-----+
| 7 |
+-----+
1 row in set (0.23 sec)
```

FONCTION GROUP_CONCAT (expr)

Cette fonction renvoie un résultat sous forme de chaîne avec les valeurs concaténées d'un groupe.

La syntaxe complète est la suivante:

```
GROUP_CONCAT([DISTINCT] expr [,expr ...]
[ORDER BY {unsigned_integer | col_name | expr}
[ASC | DESC] [,col ...]]
[SEPARATOR str_val])
```

GROUP_CONCAT () a été ajouté dans **MySQL 4.1**.

Exemple:

```
mysql> SELECT student_name,
-> GROUP_CONCAT(test_score)
-> FROM student
-> GROUP BY student_name;
```

```
mysql> SELECT NOM, GROUP_CONCAT(NOTES) FROM NOTES_ETUDIANT GROUP BY NOM;
+-----+-----+
| NOM          | GROUP_CONCAT(NOTES) |
+-----+-----+
| KIWOZOU ALAIN | 10,12                |
| MANDZOU SILVAIN | 13,16                |
| NGOMA PIERRE   | 13,16                |
| OBOA GEORGE    | 14,11                |
| YOLOU FRANCOIS | 15,13                |
+-----+-----+
5 rows in set (0.00 sec)
```

FONCTION MIN (expr), MAX (expr)

Renvoie la valeur **minimale** ou **maximale** de expr. **MIN ()** et **MAX ()** peuvent prendre un argument de chaîne; dans ce cas, ils renvoient la valeur de chaîne minimale ou maximale.

Exemple:

```
mysql> SELECT student_name, MIN(test_score), MAX(test_score)
-> FROM student
-> GROUP BY student_name;
```

```
mysql> SELECT NOM, MIN(NOTES), MAX(NOTES) FROM NOTES_ETUDIANT GROUP BY NOM;
+-----+-----+-----+
| NOM          | MIN(NOTES) | MAX(NOTES) |
+-----+-----+-----+
| KIWOZOU ALAIN |          10 |          12 |
| MANDZOU SILVAIN |          13 |          16 |
| NGOMA PIERRE  |          13 |          16 |
| OBOA GEORGE  |          11 |          14 |
| YOLOU FRANCOIS |          13 |          15 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

FONCTION STD (expr), STDDEV (expr)

Renvoie l'écart type de expr (la racine carrée de **VARIANCE ()**). Ceci est une extension du **SQL** standard. La forme **STDDEV ()** de cette fonction est fournie pour la compatibilité **Oracle**.

FONCTION SUM (expr)

Retourne la somme de expr. Notez que si le jeu de retour n'a pas de lignes, il retourne **NULL**!

FONCTION VARIANCE (expr)

Renvoie la variance standard de expr (en considérant les lignes comme la population entière, et non comme un échantillon; le nombre de lignes est donc un dénominateur). Il s'agit d'une extension du langage **SQL** standard, disponible uniquement dans **MySQL 4.1** ou ultérieur.

V.2. MODIFICATEURS GROUP BY

Depuis **MySQL 4.1.1**, la clause **GROUP BY** autorise un modificateur **WITH ROLLUP** qui entraîne l'ajout de lignes supplémentaires à la sortie du résumé. Ces lignes représentent des opérations récapitulatives de niveau supérieur (ou super agrégées). **ROLLUP** vous permet donc de répondre à des questions à plusieurs niveaux d'analyse en une seule requête. Il peut être utilisé, par exemple, pour prendre en charge les opérations **OLAP (Online Analytical Processing)**.

À titre d'illustration, supposons qu'un tableau nommé sales comporte des colonnes année, pays, produit et bénéfice permettant d'enregistrer la rentabilité des ventes:

```
mysql> CREATE TABLE VENTE (ANNEE INT NOT NULL, PAYS VARCHAR(20) NOT NULL, PRODUIT VARCHAR(32) NOT NULL, BENEFICE INT);
Query OK, 0 rows affected (0.11 sec)
```

Le contenu de la table peut être résumé chaque année avec un simple **GROUP BY** comme ceci:

```
mysql> SELECT * FROM VENTE;
```

ANNEE	PAYS	PRODUIT	BENEFICE
2001	CONGO	TUBERCULES	200850
2003	GHANA	HUILE DE PALME	108000
2008	COTE D'IVOIRE	CHOCOLAT	1780850
2010	ARABIE SAUDITE	PETROLE	2147483647
2018	NIGERIA	CIMENT	2147483647
2001	SOUDAN	TUBERCULES	8000500
2018	RDC	CIMENT	39560850
2010	LIBYE	PETROLE	2147483647
2008	GHANA	CHOCOLAT	65480850
2003	CAMEROUNE	HUILE DE PALME	5678000

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT ANNEE,SUM(BENEFICE) FROM VENTE GROUP BY ANNEE;
```

ANNEE	SUM(BENEFICE)
2001	8201350
2003	5786000
2008	67261700
2010	4294967294
2018	2187044497

```
5 rows in set (0.00 sec)
```

Cette sortie affiche le bénéfice total pour chaque année, mais si vous souhaitez également déterminer le total des bénéfices pour toutes les années, vous devez additionner vous-même les valeurs individuelles ou exécuter une requête supplémentaire.

Vous pouvez également utiliser **ROLLUP**, qui fournit les deux niveaux d'analyse avec une seule requête. L'ajout d'un modificateur **WITH ROLLUP** à la clause **GROUP BY** entraîne la génération d'une autre ligne indiquant le total général pour toutes les valeurs de l'année:

```
mysql> SELECT ANNEE,SUM(BENEFICE) FROM VENTE GROUP BY ANNEE WITH ROLLUP;
```

ANNEE	SUM(BENEFICE)
2001	8201350
2003	5786000
2008	67261700
2010	4294967294
2018	2187044497
NULL	6563260841

```
6 rows in set (0.00 sec)
```

La ligne de super-agrégat du total général est identifiée par la valeur **NULL** dans la colonne année.

ROLLUP a un effet plus complexe lorsqu'il existe plusieurs colonnes **GROUP BY**. Dans ce cas, chaque fois qu'il y a une «rupture» (modification de valeur) dans une colonne de regroupement sauf la dernière, la requête génère une ligne de résumé super-agrégée supplémentaire.

Par exemple, sans **ROLLUP**, un résumé du tableau des ventes basé sur l'année, le pays et le produit pourrait ressembler à ceci:

```
mysql> SELECT ANNEE,PAYS,PRODUIT,SUM(BENEFICE) FROM VENTE GROUP BY ANNEE,PAYS,PRODUIT;
```

ANNEE	PAYS	PRODUIT	SUM(BENEFICE)
2001	CONGO	TUBERCULES	200850
2001	SOUDAN	TUBERCULES	8000500
2003	CAMEROUNE	HUILE DE PALME	5678000
2003	GHANA	HUILE DE PALME	108000
2008	COTE D'IVOIRE	CHOCOLAT	1780850
2008	GHANA	CHOCOLAT	65480850
2010	ARABIE SAOUDITE	PETROLE	2147483647
2010	LIBYE	PETROLE	2147483647
2018	NIGERIA	CIMENT	2147483647
2018	RDC	CIMENT	39560850

10 rows in set (0.00 sec)

- ❖ La sortie indique des valeurs récapitulatives uniquement au niveau de l'analyse **année / pays / produit**.

- ❖ Lorsque **ROLLUP** est ajouté, la requête génère plusieurs lignes supplémentaires:

```
mysql> SELECT ANNEE,PAYS,PRODUIT,SUM(BENEFICE) FROM VENTE GROUP BY ANNEE,PAYS,PRODUIT WITH ROLLUP;
```

ANNEE	PAYS	PRODUIT	SUM(BENEFICE)
2001	CONGO	TUBERCULES	200850
2001	CONGO	NULL	200850
2001	SOUDAN	TUBERCULES	8000500
2001	SOUDAN	NULL	8000500
2001	NULL	NULL	8201350
2003	CAMEROUNE	HUILE DE PALME	5678000
2003	CAMEROUNE	NULL	5678000
2003	GHANA	HUILE DE PALME	108000
2003	GHANA	NULL	108000
2003	NULL	NULL	5786000
2008	COTE D'IVOIRE	CHOCOLAT	1780850
2008	COTE D'IVOIRE	NULL	1780850
2008	GHANA	CHOCOLAT	65480850
2008	GHANA	NULL	65480850
2008	NULL	NULL	67261700
2010	ARABIE SAOUDITE	PETROLE	2147483647
2010	ARABIE SAOUDITE	NULL	2147483647
2010	LIBYE	PETROLE	2147483647
2010	LIBYE	NULL	2147483647
2010	NULL	NULL	4294967294
2018	NIGERIA	CIMENT	2147483647
2018	NIGERIA	NULL	2147483647
2018	RDC	CIMENT	39560850
2018	RDC	NULL	39560850
2018	NULL	NULL	2187044497
NULL	NULL	NULL	6563260841

26 rows in set (0.00 sec)

Pour cette requête, l'ajout de **ROLLUP** force la sortie à inclure des informations de synthèse à CINQ niveaux d'analyse, et non à un seul. Voici comment interpréter la sortie **ROLLUP**:

- ❖ Après chaque série de lignes de produits pour une année et un pays donnés, une ligne récapitulative supplémentaire est générée, indiquant le total pour tous les produits. La colonne produit de ces lignes est définie sur **NULL**.
- ❖ Après chaque série de lignes d'une année donnée, une ligne récapitulative supplémentaire est générée. Elle affiche le total pour tous les pays et produits. Les colonnes pays et produits de ces lignes sont définies sur **NULL**.
- ❖ Enfin, après toutes les autres lignes, une ligne récapitulative supplémentaire est générée. Elle indique le total général pour toutes les années, tous les pays et tous les produits. Cette ligne contient les colonnes année, pays et produits définies sur **NULL**.

QUESTIONS D'AUTO-EVALUATION

5. Les fonctions d'agrégation **SUM ()** et **AVG ()** ne fonctionnent pas avec les valeurs ____.

VI. RESUME

Cette unité couvrait diverses fonctions de recherche textuelles, de casting, de cryptage et d'information. Il traite ensuite de diverses fonctions et modificateurs pouvant être utilisés avec la clause **GROUP BY**.

1. Fonctions de recherche de texte intégral: Ces fonctions sont utilisées pour effectuer une recherche de texte intégral ou des fonctions basées sur des chaînes. Ces fonctions incluent les recherches booléennes, les recherches d'extension de requête, etc.
2. Fonctions de conversion: Les fonctions **CAST ()** et **CONVERT ()** peuvent être utilisées pour prendre une valeur d'un type et produire une valeur d'un autre type. Ces fonctions sont principalement utilisées pour effectuer des conversions entre différents types de données.
3. Autres fonctions: Les autres fonctions comprennent les fonctions **Bit**, **Cryptage** et **Information**. Ils sont utilisés pour travailler sur les manipulations de bits, le cryptage des données et la collecte d'informations à partir des sources de données.
4. Group by Clauses: La clause Group By est essentiellement utilisée avec les fonctions d'agrégation telles que **SUM**, **AVERAGE**, etc. Ils peuvent également être utilisés pour modifier la sortie d'une requête en fonction des besoins.

QUESTIONS SUR LES TERMINAUX

1. Décrivez les opérateurs prenant en charge les recherches booléennes en texte intégral.
2. Expliquez quelques fonctions de cryptage.